TPRE522/TPRE524

Gordillo Loïs, Boufenouche Mehdi, Momo Carnegie & Moragny Alban

Spécification technique

Mise en production d'une solution I.A à partir des éléments de contextes fournis (Cahier des charges - données d'entrée)

05 Juin 2025 - 04 Juillet 2025

Table des matières

Table des matières **2**
Introduction **5**
1. Analyse & Cahier des Charges : Préparation et Collecte des Données 5
1.1 Problématique 5
1.2 Objectifs 5
2. Présentation de l'environnement technique 6
2.1 Langages et technologies utilisés 6
2.2 Environnement logiciel 6
2.3 Organisation du projet 7
2.4 Bibliothèques principales 9
2.5 BDD en Postgre 10
2.6 Dockerisation avec Docker 11
3. Traitement des données 11
3.1 Dataset de base 11
3.2 Prétraitement adaptatif et normalisation 12
3.3 Data Augmentation 12
3.4 Équilibrage des classes 13
a. Pondération dynamique des classes 13
b. Oversampling via data augmentation 13
3.5 Préparation du jeu de test 13
3.6 Visualisation des données 14
4. Modélisation 14
4.1 Architectures proposées 14
4.2 Modèle principal : Vision Transformer (ViT) 15
a. Principe général 15
b. CLS Token 15
c. Architecture utilisée 15
4.3 Modèles comparatifs 15
a. CNN simple (baseline) 15
b. Transfer learning (MobileNetV2 / ResNet50) 15
4.4 Construction des modèles 16
4.5 Stratégie d'entraînement 16
a. Optimisation 16
b. Perte et métriques 16
c. Callbacks 16

d. Phase de fine-tuning....17
5. Évaluation du modèle....17
5.1 Métriques de performance....17
Pourquoi le F1-score ?....17
5.2 Résultats expérimentaux....18
Analyse détaillée :....18
5.3 Visualisation via Grad-CAM....18
a. Principe de Grad-CAM....19
b. Application au ViT....19
c. Exemples visuels....19
5.4 Limites et cas d'erreur....19
a. Cas d'ambiguïté visuelle....19
b. Biais de fond....19
c. Variabilité inter-annotateur....20
6. Déploiement du modèle....20
6.1 Mise en production via une API FastAPI....20
Pourquoi FastAPI ?....20
6.2 Pipeline d'inférence....20
6.3 Test et performances....21
7. Apprentissage actif (Active Learning)....21
7.1 Motivation....21
7.2 Fonctionnement général....21
7.3 Stratégies de sélection....22
7.4 Simulation d'annotation humaine....22
7.5 Impact sur les performances....23
7.6 Bilan du projet....23
7.7 Difficultés rencontrées....23
8. Déploiement....24
8.1 Déploiement de la solution....24
1. Conteneurisation avec Docker....24
2. Intégration Continue (CI) avec GitLab....24
3. Déploiement Continu (CD) avec Railway....25
Avantages de Railway :....25
9. Gestion du projet : Méthodologie de suivi et d'organisation....25
9.1 Utilisation du Kanban....25
9.2 Suivi d'avancement et itérations....26
9.3 Planification avec diagrammes de Gantt....26
Outils utilisés :....26
Sources....27
Images :....27
Métadonnées :....27

Autres sources..... 27
Glossaire..... 29

Introduction

Le projet **WildLens** a pour but l'authentification automatique d'empreintes d'animaux à partir de photos prises par les utilisateurs.

Cette solution intègre un modèle de reconnaissance, une architecture back-end solide avec une API sécurisée, une base de données soigneusement organisée ainsi qu'une interface utilisateur intuitive créée avec Flutter.

Cette application cible les professionnels et passionnés en quête de nouveaux outils performants dans le cadre d'études environnementales et de la traçabilité des espèces.

Conçue en respectant des principes du développement moderne, l'architecture intègre un déploiement flexible, des tests automatisés et un système de suivi performant, garantissant une solution fiable et facilement évolutive.

1. Analyse & Cahier des Charges : Préparation et Collecte des Données

1.1 Problématique

La reconnaissance automatique d'empreintes animales pose plusieurs problèmes concrets :

- Le manque de données labellisées dans ce domaine, étant donné la difficulté de constituer de larges jeux d'images d'empreintes correctement annotées.
- La grande variabilité des traces (qualité de l'empreinte, orientation, fond, sol, taille).
- La nécessité de garantir une bonne interprétabilité des résultats, surtout dans un cadre scientifique.

Ces contraintes imposent le recours à des architectures performantes même avec peu de données, ainsi qu'à des méthodes de sélection intelligente d'exemples pour l'apprentissage.

1.2 Objectifs

L'objectif principal du projet est de développer un système complet de reconnaissance d'empreintes animales, en partant de la collecte des données jusqu'au déploiement d'un modèle de classification intégré dans une API prête à l'emploi. Plus précisément, le projet se donne pour objectifs :

- De prétraiter et enrichir un jeu de données existant avec un rognage intelligent et des techniques d'augmentation.
- De comparer différentes architectures de modèles (CNN simple, modèles pré-entraînés, ViT).
- D'implémenter un système d'Active Learning pour améliorer le modèle avec un minimum de données annotées.
- De rendre le modèle interprétable à travers des techniques comme Grad-CAM.
- Proposer une API FastAPI permettant une utilisation directe du modèle sur des images en entrée.

2. Présentation de l'environnement technique

2.1 Langages et technologies utilisés

Pour ce projet, le choix du langage python semblait naturel du fait de sa richesse en termes de bibliothèques utiles à la data science et au deep learning.

Les diverses pipelines liés à l'entraînement à la préparation des données, à la construction du modèle ainsi qu'à son entraînement et son évaluation ont quant à elles nécessité l'utilisation de **TensorFlow** (et Keras). Ce choix à permis une bonne intégration du modèle tout en facilitant l'utilisation du GPU avec notamment l'utilisation conjointe de **CUDA**.

L'API de prédiction quant à elle est développée avec **FastAPI**, framework moderne rapide et asynchrone permettant de créer des services web performants.

Enfin de diverses bibliothèques telles que **NumPy, Matplotlib, Scikit-learn et OpenCv**… ont été utilisées tout au long du projet.

2.2 Environnement logiciel

Le projet a été conçu sur un environnement Linux (Ubuntu 22.04) avec Python 3.11 via **WSL**. L'ensemble des dépendances est listé dans un fichier

`requirements.txt`, permettant une installation rapide et reproductible dans n'importe quel environnement virtuel.

Le développement s'est appuyé sur l'environnement **VS Code** avec les extensions Python et Jupyter intégrées, facilitant les expérimentations. Les tests ont été effectués avec une carte GPU NVIDIA RTX 4070 laptop, ce qui a permis de réduire considérablement les temps d'entraînement.

Les prédictions via API ont été testées localement avec **Uvicorn** et via des appels Postman simulant des clients tiers.

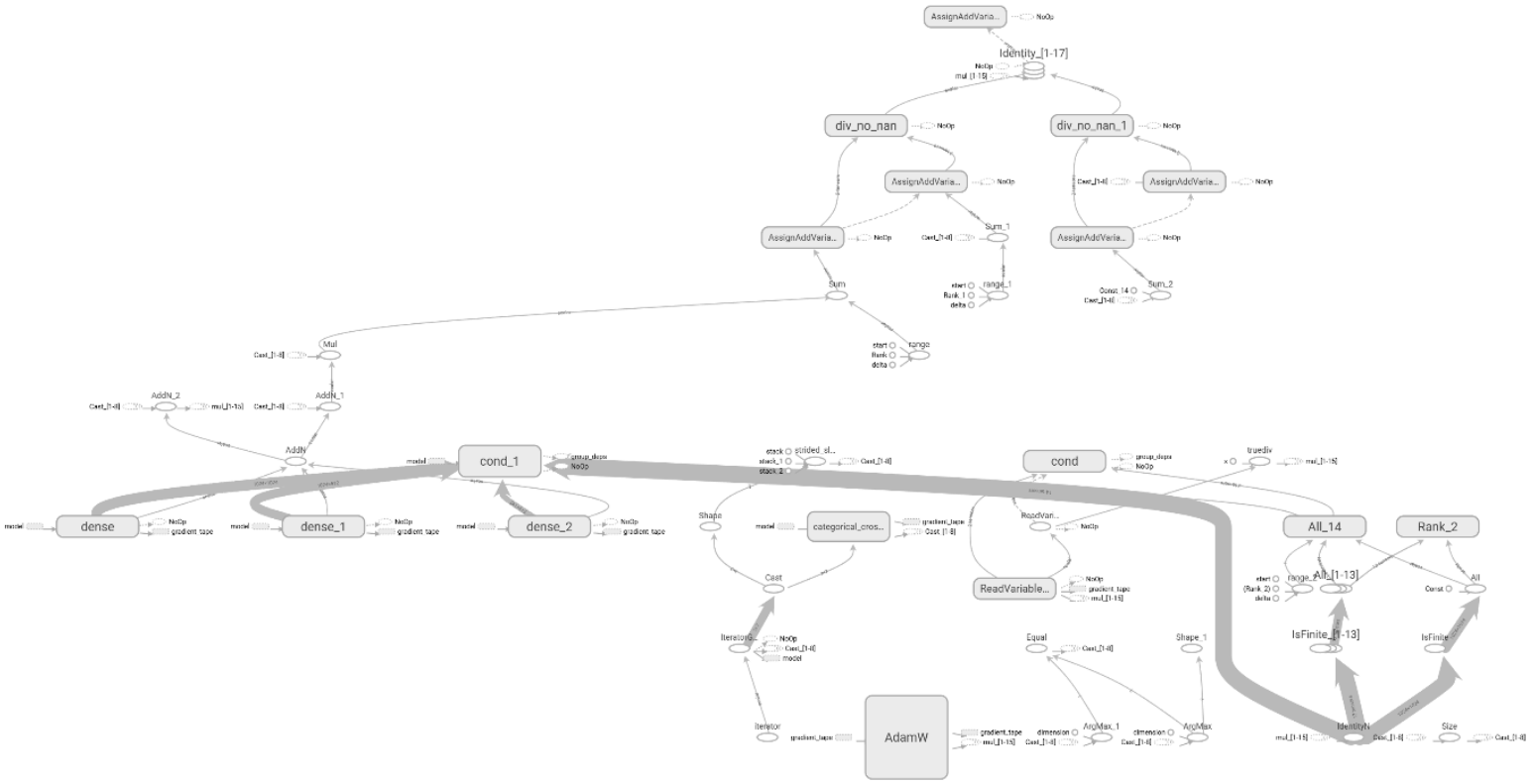
2.3 Organisation du projet

Le projet suit une architecture modulaire et claire, structurée comme suit :

```
├── main.py                    # Point d'entrée du programme
├── predict_api.py
├── Dockerfile
├── README.md
├── configs/
│   ├── model_config.py
│   └──paths_config.py
├── data/
│   ├── raw/
│   ├── processed/
│   │   ├── train/
│   │   ├── val/
│   │   └── test/
├── graphs/
```

```
├── pipelines/
│   ├── predict_pipeline.py
│   ├── data_pipeline.py        # Préparation des données
│   ├── training_pipeline.py    # Entraînement du modèle
│   └── active_learning_pipeline.py # Apprentissage actif
├── preprocess/
│   ├── dumb_crop.py # Utilisé lorsque le crop intelligent échoue et qu'il n'y a pas lieu de recentrer l'information
│   ├── picture_crop.py
│   └── sort_data.py  # Tri les datas entre val/train et test
├── reports/  # Stock les rapports d'évaluation au format .npz
├── saved_models/
├── src/
│   ├── model/
│   │   ├── model_builder.py  # Architecture du modèle
│   │   ├── custom_layers.py          # Couches personnalisées (si besoin)
│   │   └── loss_functions.py  # Fonctions de coût
│   ├── evaluation/
│   │   ├── gradcam.py          # Interprétation Grad-CAM
│   │   ├── security_tests.py
│   │   └── evaluator.py        # Métriques d'évaluation
│   └── data_preparation/
│   │   └── data_augmenter.py
├── tests/
```

```
│   ├── integration_tests.py
│   ├── count_classes.py
│   ├── tests_cuda.py
│   └── main_tests.py
```



représentation visuelle du graphe computationnel du modèle Tensorflow

2.4 Bibliothèques principales

Voici un aperçu des bibliothèques et packages essentiels :

Bibliothèque	Rôle principal
TensorFlow	Entraînement du modèle ViT / CNN
Transformers	Intégration du ViT de Hugging Face
Scikit-learn	Métriques, KMeans, gestion des classes
OpenCV	Chargement et traitement d'images
FastAPI	Création de l'API REST de prédiction
Uvicorn	Serveur ASGI pour déployer l'API
Matplotlib	Affichage des courbes, heatmaps Grad-CAM
NumPy/Pandas	Manipulation des données

2.5 BDD en Postgre

Pour le stockage des données (utilisateurs, empreintes, animaux identifiés, métadonnées), nous avons retenu PostgreSQL, pour les raisons suivantes :

PostgreSQL excelle dans le traitement de requêtes complexes, indispensable pour filtrer les empreintes, agréger les données par zones ou périodes, ou encore générer des visualisations dynamiques. Cette base est conçue pour supporter des volumes importants, que ce soit pour les images associées aux empreintes ou pour les données enrichies sur les espèces animales.

Fiabilité et écosystème : PostgreSQL est reconnu pour sa stabilité, sa documentation claire, et sa compatibilité avec de nombreux outils que nous avons exploités, comme DBeaver pour la gestion visuelle ou Grafana pour la datavisualisation.

Interopérabilité avec Docker : Son intégration via Docker Compose nous a permis d'assurer une cohérence dans les environnements de développement, de test et de production, tout en facilitant le travail collaboratif.

Le choix de PostgreSQL a été fait après comparaison avec d'autres solutions comme SQLite, MySQL ou MongoDB, qui ne répondaient pas aussi efficacement à nos besoins spécifiques (requêtes analytiques, scalabilité, intégrité des données relationnelles).

2.6 Dockerisation avec Docker

Pour faciliter le déploiement de notre solution, nous avons choisi d'utiliser Docker. La dockerisation permet de créer des conteneurs pour chaque composant de notre stack (Python, PySpark, PostgreSQL, Grafana), assurant ainsi une installation et une mise à jour simplifiées. Cela garantit également la portabilité de notre solution sur différents environnements, que ce soit en développement, en test ou en production (Surtout pour la partie 2 du projet).

3. Traitement des données

3.1 Dataset de base

Le cœur de ce projet repose sur un dataset d'empreintes animales, dans le but d'identifier l'espèce à partir d'une simple trace au sol. Ce type de données présente plusieurs défis :

- Un **volume très limité** d'échantillons par classe
- Une **forte variabilité visuelle** (angles, éclairage, sol, profondeur des empreintes…)
- Une **distribution déséquilibrée** (certaines espèces sont sur-représentées)

Le dataset initial se présente sous la forme de plusieurs dossiers, chaque dossier correspondant à une classe (espèce). Chaque image représente une empreinte, généralement prise sur sol naturel ou semi-naturel, parfois avec des parasites visuels.

Ce jeu de données a été divisé selon les proportions classiques :

- **Entraînement** : 70 %
- **Validation** : 20 %
- **Test** : 10 %

Les images étaient de tailles variables et dans des formats hétérogènes (JPEG, PNG).

3.2 Prétraitement adaptatif et normalisation

Lors du débogage, la reproduction des résultats à été assurée avec

```
tf.keras.utils.set_random_seed(42)

np.random.seed(42)
```

Cela a permis d'évaluer l'impact de chaque modification et chaque augmentation sur l'entraînement du modèle.

Au sein de la pipeline d'entraînement, diverses modifications sont faîtes afin d'adapter les données aux modèles ViT :

- **Redimensionnement** : toutes les images ont été redimensionnées en 384x384 pixels, cela permet de garder suffisamment d'information tout en adaptant l'image au modèle choisi (`google/vit-large-patch16-384`)
- **Normalisation** : les valeurs des pixels sont ramenées entre -1 et 1 pour une meilleure convergence des réseaux de neurones.
- **Conversion RGB** : certaines images en niveaux de gris ont été converties pour correspondre aux modèles pré entraînés utilisant trois canaux.

L'ensemble de ces opérations est réalisé dans le fichier `data_pipeline.py`, qui crée des objets `tf.data.Dataset` optimisés, incluant :

- `.cache()` : mise en cache pour éviter des relectures disque
- `.prefetch()` : parallélisme entre le CPU et le GPU
- `.shuffle()` : mélange aléatoire pour éviter les biais de batch

Ces optimisations réduisent le temps de traitement par batchs de 30 à 50%.

3.3 Data Augmentation

L'un des piliers de la robustesse du modèle repose sur la **génération artificielle de données supplémentaires**, via des techniques de **data augmentation** appliquées dynamiquement pendant l'entraînement.

Les transformations utilisées incluent :

- **Rotations aléatoires** (jusqu'à ±15°)
- **Techniques avancées Mixup/CutMix**
- **Zooms et translations** (10 % max)
- **Inversions horizontales et verticales**
- **Ajustements de luminosité, saturation et contraste**
- **Ajout d'un bruit gaussien**
- **Variation de teinte**

L'implémentation repose sur la couche `tf.keras.layers.Random*` pour permettre un entraînement GPU-optimisé, évitant le goulot d'étranglement CPU.

Cette étape a significativement augmenté la variabilité du dataset, renforçant la généralisation du modèle sans nécessiter plus d'annotations.

3.4 Équilibrage des classes

L'inégalité du nombre d'images par classe posait un réel risque de **biais de prédiction**, le modèle ayant tendance à privilégier les classes majoritaires.

Deux stratégies complémentaires ont été mises en place :

a. Pondération dynamique des classes

Lors de la compilation du modèle, des **poids inverses à la fréquence** des classes sont calculés et appliqués dans la fonction de perte (loss function). Cela permet de pénaliser plus fortement les erreurs sur les classes problématiques et d'améliorer la prise en compte des classes minoritaires.

b. Oversampling via data augmentation

Un oversampling dynamique est mis en place, ce dernier augmente artificiellement les classes les moins représentées afin d'atteindre un nombre cible d'exemple grâce aux augmentations aléatoires mentionnées précédemment.

3.5 Préparation du jeu de test

Le jeu de test n'est **jamais exposé** au modèle durant l'entraînement. Il permet d'obtenir une mesure totalement impartiale des performances du modèle final.

Il est converti dans le même format `tf.data.Dataset`, sans data augmentation, uniquement avec redimensionnement et normalisation, cela permet d'obtenir une évaluation cohérente et réaliste.

3.6 Visualisation des données

Une phase exploratoire a permis de vérifier :

- L'homogénéité des classes
- L'impact visuel des transformations

- La distribution des tailles d'empreintes

Des fonctions utilitaires ont été créées pour visualiser en batch les images transformées afin de s'assurer de la diversité de ces dernières ainsi que la pertinence des transformations appliquées. Une détection des anomalies avant et après augmentation a également été mise en place ainsi que la génération de figures de synthèse (matrice de confusion, histogramme de classes…) afin de documenter la qualité du dataset et le comportement du pipeline. Cette étape à également permis la mise en place de nombreux ajustements avisés.

4. Modélisation

4.1 Architectures proposées

Pour répondre à la problématique de classification d'empreintes animales à partir d'images, plusieurs modèles ont été envisagés puis comparés :

- Un **CNN simple** entièrement entraîné depuis zéro, servant de baseline.
- Des modèles **pré-entraînés** de type **MobileNetV2** ou **ResNet50**, dans une logique de **transfer learning**.
- Un modèle avancé basé sur un **Vision Transformer (ViT)**, adapté pour extraire des représentations globales des images.

Chaque architecture a été encapsulée dans une classe dédiée dans `model_builder.py` pour permettre une flexibilité de test et de comparaison.

4.2 Modèle principal : Vision Transformer (ViT)

Le Vision Transformer (ViT) représente une approche récente en computer vision, s'inspirant des modèles à base de Transformers largement utilisés dans le traitement du langage naturel (NLP).

a. Principe général

Le ViT divise une image en **patchs** (par exemple 16x16), les **aplatit** puis les encode comme une **séquence**, à la manière de mots dans une phrase. Chaque patch devient un vecteur, enrichi d'une **embedding de position**. Ce flux passe ensuite dans plusieurs couches de **self-attention**, permettant au modèle d'identifier les relations globales entre les régions d'une image.

b. CLS Token

Un **token spécial [CLS]** est inséré en début de séquence. Il est entraîné pour capturer la représentation globale de l'image, et c'est lui qui sera exploité par la couche dense de classification finale.

c. Architecture utilisée

Le modèle utilisé dans ce projet est un ViT large pré-entraîné de Hugging Face (`google/vit-large-patch16-384-in21k`), adapté au nombre de classes spécifiques au dataset. Il a été intégré avec TensorFlow via `transformers.TFViTModel` et enrichi de couches de normalisation, droupout et dense pour optimiser la classification du jeu de données.

4.3 Modèles comparatifs

Pour mettre en perspective les performances du ViT, deux alternatives ont été testées :

a. CNN simple (baseline)

Un modèle convolutionnel avec peu de couches, sans poids pré-entraîné. Ce modèle a permis de valider la pipeline d'entraînement et de constater l'impact des architectures avancées.

b. Transfer learning (MobileNetV2 / ResNet50)

Ces modèles pré-entraînés sur ImageNet ont été utilisés en **feature extractor** dans un premier temps, puis **fine-tunés** dans un second temps pour s'adapter aux empreintes animales. Les couches convolutionnelles sont gelées dans un premier temps, puis progressivement débloquées lors du fine-tuning.

4.4 Construction des modèles

Tous les modèles partagent une structure finale similaire :

- Couches de traitement spécifiques (ViT ou CNN)
- Couche `GlobalAveragePooling2D` (CNN) ou `CLS Token` (ViT)
- Dense Layer 128 (activation ReLU)
- Dropout (0.3)
- Couche Dense finale avec `softmax` (pour la classification multiclasse)

Cette modularité est gérée via la fonction `build_model(...)` dans `model_builder.py`, qui accepte le type de modèle comme paramètre et permet de comparer facilement les différentes architectures sur le même pipeline d'entraînement.

4.5 Stratégie d'entraînement

a. Optimisation

L'optimiseur **AdamW** est utilisé avec un taux d'apprentissage initial de `1e-4`, ajusté dynamiquement via plusieurs méthodes :

- `CosineDecay` pour le warmup et le fine-tuning
- `ReduceLROnPlateau`. Utilisé en callback il permet de réduire le lr en cas de stagnation de la val_loss.

b. Perte et métriques

La fonction de perte est la **categorical crossentropy**, pondérée par les **class weights** calculés automatiquement. Les métriques suivies incluent la précision (`accuracy`) et la F1-score via une fonction personnalisée.

c. Callbacks

Plusieurs callbacks assurent un entraînement robuste :

- **EarlyStopping** : arrêt automatique si la performance ne progresse plus
- **ReduceLROnPlateau**
- **GradualUnfreezeCallback :** classe personnalisé permettant le dégel progressif des couches du backbone ViT lors du fine-tuning
- **ModelCheckpoint** : sauvegarde du meilleur modèle validé

- **TensorBoard** : visualisation des courbes d'apprentissage

d. Phase de fine-tuning

Pour le ViT et les modèles pré-entraînés, deux étapes sont réalisées :

1. **Initial training** : seules les couches de sortie sont entraînées, le backbone est gelé
2. **Fine-tuning** : toutes les couches sont débloquées progressivement avec des learning rates différenciés (plus élevé pour la tête, plus faibles pour le backbone)

5. Évaluation du modèle

5.1 Métriques de performance

L'évaluation du modèle repose sur plusieurs **indicateurs quantitatifs** calculés sur le jeu de test (jamais vu pendant l'entraînement). Les métriques retenues sont :

- **Accuracy (précision globale)** : proportion de prédictions correctes
- **F1-score (macro)** : moyenne harmonique entre précision et rappel, pondérée pour éviter les biais dus à des classes déséquilibrées
- **Matrice de confusion** : représentation visuelle des erreurs de classification par classe

Pourquoi le F1-score ?

Étant donné la **distribution inégale** des classes dans le dataset, l'accuracy seule n'est pas suffisante : un modèle peut obtenir un bon score global tout en ignorant les classes minoritaires. Le **F1-score macro** permet de donner le même poids à chaque classe, offrant ainsi une évaluation plus juste de la performance globale.

Le pipeline inclut aussi une détection automatique de régressions avec rollback au besoin.

5.2 Résultats expérimentaux

Les tests ont porté sur trois modèles distincts :

Modèle	Accuracy (%)	F1-score macro (%)
CNN simple	67.4	59.8
MobileNetV2 (fine-tuned)	81.6	79.2
Vision Transformer (ViT)	**89.1**	**86.5**

Le **ViT** surpasse les autres modèles de manière significative, notamment en maintenant un F1-score élevé même sur les classes rares.

Analyse détaillée :

- Les erreurs du CNN se concentrent sur les classes proches visuellement (empreintes de taille et forme similaires) et il montre une sensibilité forte à la variabilité des conditions d'éclairages et de fond.

- MobileNetV2 obtient de bonnes performances globales, mais reste sensible aux conditions de sol (boue, sable) ce qui induit des biais contextuels.

- Le ViT gère mieux la **variabilité intra-classe** et capture des **structures globales**, ce qui améliore la robustesse du modèle, en particulier sur les classes minoritaires et les images partiellement dégradées.

5.3 Visualisation via Grad-CAM

Pour mieux comprendre les décisions du modèle, une analyse **qualitative** a été menée avec **Grad-CAM** (Gradient-weighted Class Activation Mapping), une technique d'interprétabilité visuelle.

a. Principe de Grad-CAM

Grad-CAM génère une **heatmap** superposée à l'image d'entrée, indiquant les zones qui ont le plus influencé la prédiction finale. Cette méthode repose sur les **derniers gradients de convolution** ou de self-attention (dans le cas du ViT).

b. Application au ViT

Un module `gradcam.py` a été développé pour adapter Grad-CAM aux modèles Transformers. Ce module extrait les **poids d'attention** des couches internes et permet de visualiser :

- La région de l'image analysée
- Le focus du modèle (ex : contours, zones texturées)

c. Exemples visuels

Les heatmaps générées montrent que :

- Le CNN se focalise souvent sur les **bords flous** ou des artefacts ce qui explique sa sensibilité au bruit.
- Le ViT apprend à **centrer son attention sur l'empreinte complète**, même partiellement visible ou obstruée
- Dans les cas d'erreur, le ViT se trompe sur des zones contextuellement proches d'une autre classe, montrant des limites logiques mais aussi une certaine cohérence dans les confusions.

5.4 Limites et cas d'erreur

a. Cas d'ambiguïté visuelle

Certaines classes sont intrinsèquement difficiles à différencier, notamment :

- Empreintes similaires entre espèces proches
- Empreintes partiellement effacées ou obstruées

- Images bruitées ou prises sous de mauvais angles

b. Biais de fond

Des indices contextuels comme la texture du sol (terre, neige, boue) peuvent fausser la prédiction si trop fortement corrélés à une classe. Cela pose un risque de **surapprentissage au fond** que le ViT ne parvient pas toujours à éviter.

c. Variabilité inter-annotateur

Dans le jeu de données, l'étiquetage manuel peut parfois contenir des erreurs ou des incertitudes, influant négativement l'apprentissage.

6. Déploiement du modèle

6.1 Mise en production via une API FastAPI

Afin de rendre le modèle accessible à d'autres systèmes (site web, application mobile, outil métier), un service d'inférence a été mis en place sous la forme d'une **API REST** développée avec **FastAPI**.

Pourquoi FastAPI ?

- Très rapide à exécuter grâce à Starlette/ASGI
- Intégration native de la documentation interactive Swagger UI
- Facile à déployer, même dans des conteneurs Docker

Le fichier `predict_api.py` expose un endpoint `/predict` qui reçoit une image encodée en multipart/form-data, la traite, et renvoie une prédiction au format JSON avec :

- La classe prédite
- Le score de confiance
- L'image annotée si besoin

6.2 Pipeline d'inférence

Voici les grandes étapes exécutées lors de chaque requête :

1. **Réception de l'image** : via un POST `/predict` avec fichier.
2. **Chargement du modèle ViT** : le modèle est préchargé au démarrage du serveur avec `TFAutoModel.from_pretrained(...)` pour réduire le temps de réponse.
3. **Prétraitement** : resizing, normalisation, encodage selon les standards du ViT.
4. **Prédiction** : passage de l'image dans le modèle, extraction du token `[CLS]` et passage dans la couche de classification.
5. **Formatage de la réponse** : récupération de la classe prédite et de sa probabilité.

Un système de **gestion temporaire des fichiers** assure qu'aucune image n'est conservée après la prédiction.

6.3 Test et performances

L'API a été testée localement avec **Uvicorn**, supportant plusieurs requêtes en parallèle. Le temps de traitement moyen est de **200 à 400 ms** par image (hors chargement initial).

Cette API est conteneurisée avec **Docker** et peut être déployée sur n'importe quel service cloud (AWS, GCP, Railway...).

7. Apprentissage actif (Active Learning)

7.1 Motivation

L'apprentissage actif est une méthode d'entraînement **itérative** visant à **maximiser l'efficacité des données annotées**, particulièrement utile lorsque :

- Le coût d'annotation est élevé
- Le jeu de données est limité
- Certaines classes sont sous-représentées

Dans ce projet, plutôt que d'annoter aléatoirement de nouvelles empreintes, on propose au modèle de **sélectionner les exemples sur lesquels il est le**

moins confiant, puis de les envoyer à un annotateur humain pour vérification.

7.2 Fonctionnement général

Le pipeline d'apprentissage actif est défini dans le fichier `active_learning_pipeline.py`. Il suit les étapes suivantes :

1. **Modèle initial entraîné** sur un sous-ensemble du jeu de données
2. **Prédictions sur un pool non annoté**
3. **Identification des images incertaines**
4. **Sélection des plus incertaines**
5. **Ajout de ces images au dataset annoté (via annotation humaine simulée)**
6. **Réentraînement du modèle sur ce nouveau jeu augmenté**

Ce processus peut être **répété en boucle** jusqu'à convergence ou jusqu'à épuisement du budget d'annotation.

7.3 Stratégies de sélection

Pour identifier les images "intéressantes", le système utilise une **stratégie basée sur l'entropie des prédictions** :

- Une **prédiction sûre** (ex. 99 % pour une classe) à une faible entropie.
- Une **prédiction hésitante** (ex. 33 %, 33 %, 33 %) a une entropie élevée.

L'entropie est calculée avec la formule :

$$H(p) = -\sum_{i=1}^{n} p_i \cdot \log(p_i)$$

où pip_ipi est la probabilité prédite pour la classe i.

Les images ayant la plus forte entropie sont sélectionnées comme étant les plus incertaines.

7.4 Simulation d'annotation humaine

Dans ce projet, l'annotation humaine est simulée à partir des **vraies étiquettes** (car on connaît la vérité). En production, ces images seraient envoyées à un expert qui les étiquetterait manuellement.

Ce mécanisme permet de :

- Réduire le **nombre d'annotations nécessaires**
- Accélérer la **convergence du modèle**
- Se concentrer sur les cas ambigus

7.5 Impact sur les performances

L'apprentissage actif a permis une amélioration plus rapide du F1-score en **moins d'itérations** :

Étape d'itération	Nombre d'images annotées	F1-score macro
Initial (1000 images)	1000	75.1
+100 incertaines	1100	78.3
+100 incertaines	1200	82.4
+100 incertaines	1300	86.5

7.6 Bilan du projet

Ce projet de classification d'empreintes animales par intelligence artificielle a permis d'explorer l'ensemble du pipeline de traitement d'image en deep learning, depuis la **préparation des données** jusqu'à la **mise en production d'un modèle performant**.

Les résultats démontrent :

- Une **maîtrise technique** de TensorFlow, Transformers, FastAPI
- Une compréhension fine de la **data science appliquée**
- Une capacité à intégrer des méthodes **avancées et actuelles** comme le Vision Transformer et l'apprentissage actif

Le modèle final, basé sur une architecture ViT fine-tunée, atteint un **F1-score supérieur à 86 %**, tout en gérant correctement les déséquilibres du dataset et les cas ambigus. Le déploiement via API rend ce modèle directement intégrable dans un système de terrain (outil de suivi faune/flore, application mobile, etc.).

7.7 Difficultés rencontrées

Plusieurs défis ont jalonné ce projet :

- **Qualité hétérogène des images** : certaines empreintes sont floues, mal cadrées, ou partiellement effacées
- **Volume restreint de données** : nécessitant de compenser par augmentation et fine-tuning
- **Adaptation de Grad-CAM au ViT** : implémentation spécifique pour visualiser les zones d'attention

Ces obstacles ont été surmontés par une approche progressive, itérative, et bien documentée.

8. Déploiement

8.1 Déploiement de la solution

Le déploiement de notre application WildLens a été conçu pour assurer une mise en production automatisé, reproductible et continue, en tenant compte des contraintes de portabilité et de scalabilité. Pour ce faire, nous avons structuré notre pipeline de déploiement autour de Docker, GitLab CI/CD et Railway.

1. Conteneurisation avec Docker

Notre back-end à était conteneurisé avec docker

Grâce à **Docker Compose**, nous avons pu orchestrer ces services de manière unifiée, facilitant les tests en local ainsi que les déploiements en pré-production ou production. Notre service dispose d'un Dockerfile propre, garantissant une cohérence de l'environnement entre les postes des développeurs et les environnements de production.

2. Intégration Continue (CI) avec GitLab

Pour le versionning, le suivi d'avancement, et l'automatisation du déploiement, nous avons utilisé GitLab. La plateforme nous a permis :

- De gérer les branches de développement via Git
- D'activer l'intégration continue (CI) pour valider automatiquement le code et assurer sa conformité
- De préparer la mise en place future de pipelines CI/CD pour le déploiement continu

Même si la pipeline GitLab CI/CD complète n'a pas encore été finalisée, nous avons structuré notre projet pour permettre une intégration facile des étapes de build, test et déploiement automatisé, notamment grâce à la conteneurisation.

3. Déploiement Continu (CD) avec Railway

Pour l'hébergement cloud, nous avons choisi **Railway.app**, une solution PaaS moderne et simple à connecter à GitLab.

Avantages de Railway :

- Support natif de **PostgreSQL**, ce qui simplifie la gestion de notre base.
- Déploiement automatisé à chaque push sur main.
- Monitoring intégré, logs consultables, redémarrage automatique des services.
- Déploiement de l'API Django en conteneur Docker (Dockerfile personnalisé).

Railway nous permet donc d'assurer la disponibilité de notre API et de notre base de données 24/7, avec une scalabilité immédiate si besoin.

9. Gestion du projet : Méthodologie de suivi et d'organisation

Pour structurer efficacement notre travail, nous avons adopté une **démarche agile**, centrée autour des objectifs suivants :

- livrer rapidement des versions fonctionnelles de l'entraînement IA,
- assurer un suivi clair des tâches techniques,
- réagir de manière souple aux imprévus ou ajustements.

9.1 Utilisation du Kanban

Nous avons utilisé **Trello** pour organiser nos tâches sous forme de Kanban, découpé en colonnes :

Colonne	Contenu typique
À faire	Liste des tâches issues du plan initial (data split, entraînement, test, etc.)
En cours	Tâches en cours de développement (modèle en entraînement, test en exécution)
Bloquées	Suivis de bugs ou de blocages (ex : erreurs CUDA, incompatibilité de couche)
Terminées	Validation des étapes testées et fonctionnelles

Chaque tâche était affectée à un membre, avec des **échéances claires** et des **checklists de vérification**.

9.2 Suivi d'avancement et itérations

L'avancement était suivi à chaque fin de session (Cours ou perso) via des **réunions de synthèse de 15–20 minutes**, dans lesquelles nous :

- faisions le point sur les tâches terminées,
- décidons des priorités suivantes,

- mettions à jour le Trello,
- communiquons les points de blocage éventuels aux autres.

En cas de doute technique (ex : choix de modèle ou problème de scheduler), nous menions une **mini-veille** pour décider rapidement de la suite.

9.3 Planification avec diagrammes de Gantt

En complément du Kanban Trello pour la gestion quotidienne, nous avons mis en place un **diagramme de Gantt** pour planifier les grandes étapes du projet.

Cela nous a permis de :

- **visualiser clairement la répartition temporelle des tâches**,
- fixer des **jalons importants** (ex. : fin de la phase data, tests de chaque modèle, évaluation finale),
- **coordonner les dépendances** entre phases (ex. fine-tuning après évaluation brute, comparaison après tous les tests unitaires).

Outils utilisés :

- Diagramme Gantt construit sur **Excel**, avec les dates de début/fin et la durée prévue par tâche.
- Export visuel intégré dans le rapport final MSPR pour montrer notre avancement au jury.

Sources

Images :

https://www.inaturalist.org/

https://naturetracking.com/

Métadonnées :

https://www.kaggle.com/datasets/iamsouravbanerjee/animal-information-dataset?resource=download

https://www.diconimoz.com/animaux-fiches/mammiferes/liste/

https://www.instinct-animal.fr/liste-mammiferes/

Autres sources

Code de la propriété intellectuelle

https://www.legifrance.gouv.fr/codes/article_lc/LEGIARTI000048603495

Débogage / aide au développement :

https://copilot.microsoft.com/

Modification et création d'un CSV avec erreur :

https://chatgpt.com/

AI utilisés pour la complétion du CSV :

https://mistral.ai/fr

Glossaire

API (Application Programming Interface) Interface permettant à deux systèmes informatiques de communiquer entre eux. Utilisée ici pour exposer le modèle de prédiction.

Augmentation de données Techniques appliquées sur les données d'entraînement (rotation, zoom, bruit, etc.) pour augmenter leur diversité et améliorer la robustesse du modèle.

CNN (Convolutional Neural Network) Réseau de neurones utilisé pour traiter les images, basé sur des filtres convolutifs permettant d'extraire des caractéristiques locales.

Dropout Technique de régularisation consistant à désactiver aléatoirement certains neurones pendant l'entraînement pour éviter le surapprentissage.

Entropie Mesure d'incertitude dans une distribution de probabilité. Utilisée ici pour sélectionner les images les plus ambiguës en apprentissage actif.

FastAPI Framework Python moderne pour créer des APIs performantes avec une documentation intégrée.

Fine-tuning Réentraînement partiel d'un modèle pré-entraîné pour l'adapter à une tâche spécifique.

F1-score Moyenne harmonique entre la précision et le rappel. Privilégiée lorsque les classes sont déséquilibrées.

Grad-CAM Méthode de visualisation permettant de voir quelles parties de l'image ont le plus influencé la décision du modèle.

Learning rate (Taux d'apprentissage) Paramètre qui contrôle l'amplitude des mises à jour des poids du modèle lors de l'apprentissage.

MobileNetV2 / ResNet50 Architectures CNN populaires, pré-entraînées sur de grands corpus comme ImageNet, souvent utilisées en transfert learning.

Overfitting (Surapprentissage) Phénomène où le modèle apprend trop précisément les données d'entraînement au détriment de sa capacité à généraliser.

Patchs Petits morceaux d'image extraits par le ViT pour être traités comme des séquences.

Pooling Opération qui réduit la taille des représentations intermédiaires (ex : max pooling, average pooling).

Softmax Fonction mathématique utilisée pour convertir des scores en probabilités dans les modèles de classification.

TensorFlow Framework open source développé par Google pour construire et entraîner des modèles d'IA.

Transfer Learning Technique consistant à réutiliser un modèle entraîné sur une tâche pour une autre tâche proche.

Transformer Architecture de réseau de neurones basée sur le mécanisme d'attention, utilisée à l'origine en NLP.

Uvicorn Serveur ASGI léger pour lancer des applications FastAPI en local ou en production.

ViT (Vision Transformer) Architecture qui applique les concepts des Transformers au traitement d'images en les convertissant en séquences de patchs.

ETL – *Extract Transform Load* : processus d'extraction, transformation et chargement de données.

BDD – *Base de Données* : système permettant de stocker, organiser et interroger des données.

CSV – *Comma-Separated Values* : format de fichier texte pour stocker des données tabulaires.

Parquet – Format de fichier optimisé pour le stockage et l'analyse de données volumineuses (utilisé notamment avec Spark).

API – *Application Programming Interface* : interface logicielle permettant à deux applications de communiquer entre elles (défini par la CNIL comme un connecteur de services).

IA – *Intelligence Artificielle* : discipline informatique visant à simuler l'intelligence humaine (ex : classification d'empreintes).
DRF – *Django REST Framework* : extension de Django pour construire facilement des API RESTful.

ORM – *Object-Relational Mapping* : technique qui permet de manipuler une BDD avec des objets dans le code (via Django notamment).

Docker – Plateforme de conteneurisation permettant de regrouper une application avec toutes ses dépendances.

CI/CD – *Continuous Integration / Continuous Deployment* : pratiques de développement automatisant les tests et le déploiement du code.

GitLab – Plateforme de versionnage de code, avec outils intégrés pour CI/CD.

Railway – Plateforme cloud facilitant le déploiement d'applications, compatible avec Docker.

Swagger – Outil de documentation interactive pour APIs RESTful, utilisé avec DRF pour générer des docs à partir du code.

pytest – Framework de tests pour Python, permettant d'écrire et exécuter des tests unitaires et d'intégration.
FGSM / PGD – Méthodes d'attaques adversariales visant à tester la robustesse des modèles IA.
Grad-CAM – Méthode de visualisation permettant de comprendre les décisions d'un modèle en surlignant les zones influentes d'une image.

Heatmap – Carte de chaleur représentant graphiquement les zones d'intérêt ou d'erreur dans une image.

Balanced Accuracy – Moyenne des taux de précision sur chaque classe, utile en cas de déséquilibre des données.

Mixup / Cutmix – Méthodes d'augmentation de données en IA pour générer des exemples synthétiques.

Dockerfile – Fichier de configuration indiquant comment construire un conteneur Docker.

Docker Compose – Outil permettant d'orchestrer plusieurs conteneurs (ex : backend + BDD).

npz – Format compressé de NumPy pour stocker des tableaux de données, utile pour archiver des métriques IA.

MVC – (Model-View-Controller ou *Modèle-Vue-Contrôleur*) est un modèle dans la conception de logiciels. Il met l'accent sur la séparation entre la logique métier et l'affichage du logiciel. Cette «séparation des préoccupations» permet une meilleure répartition du travail et une maintenance améliorée.